

tm_hour: hour (0 to 23, same as RTC DS1347)
tm_mday: day of month (1 to 31, same as RTC DS1347)
tm_mon: month (0 to 11 but RTC DS1347 provides months from 1 to 12, so the value returned by RTC needs to have 1 subtracted from it)
tm_year: year (year since 1900; RTC DS1347 stores years from 0 to 99, and the driver considers the RTC valid from 2000 to 2099, so the value returned from RTC is added to 100 and as a result the offset is the year from 1900)
 First the *clock burst* command is executed on the RTC, which gives out all the date and time registers through the SPI interface, i.e., a total of 8 bytes:

```
buf[0] = DS1347_CLOCK_BURST | 0x80;
err = spi_write_then_read(spi, buf, 1, buf, 8);
if (err)
    return err;
```

Then the read date and time is stored in the Linux date and time structure of the RTC. The time in Linux is in binary format so the conversion is also done:

```
dt->tm_sec = bcd2bin(buf[0]);
dt->tm_min = bcd2bin(buf[1]);
dt->tm_hour = bcd2bin(buf[2] & 0x3F);
dt->tm_mday = bcd2bin(buf[3]);
dt->tm_mon = bcd2bin(buf[4]) - 1;
dt->tm_wday = bcd2bin(buf[5]) - 1;
dt->tm_year = bcd2bin(buf[6]) + 100;
```

After storing the date and time of the RTC in the Linux RTC date and time structure, the date and time is validated through *rtc_valid_tm* API. After validation, the validation status from the API is returned—if the date and time is valid, then the kernel will return the date and time in the structure to the user application; else it will return an error:

```
return rtc_valid_tm(dt);
```

The RTC write routine is implemented as follows.

First, the local buffer is filled with the *clock burst* write command, and the date and time passed to the driver write routine. The *clock burst* command informs the RTC that the date and time will follow this command, which is to be written to the RTC. Also, the time in RTC is in the *bcd* format; so the conversion is also done:

```
buf[0] = DS1347_CLOCK_BURST & 0xF7;
buf[1] = bin2bcd(dt->tm_sec);
buf[2] = bin2bcd(dt->tm_min);
buf[3] = (bin2bcd(dt->tm_hour) & 0x3F);
buf[4] = bin2bcd(dt->tm_mday);
buf[5] = bin2bcd(dt->tm_mon + 1);
buf[6] = bin2bcd(dt->tm_wday + 1);
```

```
/* year in linux is from 1900 i.e in range of 100
in rtc it is from 00 to 99 */
dt->tm_year = dt->tm_year % 100;
```

```
buf[7] = bin2bcd(dt->tm_year);
buf[8] = bin2bcd(0x00);
```

After this, the data is sent to the RTC device, and the status of the write is sent to the kernel as follows:

```
return spi_write_then_read(spi, buf, 9, NULL, 0);
```

Contributing to the RTC subsystem

The RTC DS1347 is a Maxim Dallas RTC. There are various other RTCs in the Maxim database and they are not supported by the Linux kernel, just like it is with various other manufacturers of RTCs. All the RTCs that are supported by the Linux kernel are present in the *drivers/rtc* directory of the kernel. The following steps can be taken to write support for the RTC in the Linux kernel.

1. Pick any RTC from the ‘Manufacturer’ (e.g., Maxim) database which does not have support in the Linux kernel (see the *drivers/rtc* directory for supported RTCs).
2. Download the datasheet of the RTC and study its features.
3. Refer to *rtc-ds1347.c* and other RTC files in the *drivers/rtc* directory in the Linux kernel and go over even this article for how to implement RTC drivers.
4. Write the support for the RTC.
5. Use *git* (see ‘References’ below) to create a patch for the RTC driver written.
6. Submit the patch by mailing it to the Linux RTC mailing list:
 - a.zummo@towertech.it
 - rtc-linux@googlegroups.com
 - linux-kernel@vger.kernel.org
7. The patch will be reviewed and any changes required will be suggested, and if everything is fine, the driver will be acknowledged and be added to the Linux tree. 

References

- [1] DS1347 datasheet, datasheets.maximintegrated.com/en/ds/DS1347.pdf
- [2] DS1347 driver file <https://git.kernel.org/cgit/linux/kernel/git/torvalds/linux.git/tree/drivers/rtc/rtc-ds1347.c>
- [3] Writing and submitting your first Linux kernel patch video, <https://www.youtube.com/watch?v=LLB/BBImJ14>
- [4] Writing and submitting your first Linux kernel patch text file and presentation, <https://github.com/gregkh/kernel-tutorial>

By: Raghavendra Chandra Ganiga

The author is an embedded firmware development engineer at General Industrial Controls Pvt Ltd, Pune. His interests lie in microcontrollers, networking firmware, RTOS development and Linux device drivers.